Measuring a Local AI Companion Through Its Own Runtime: Benchmark Defects, Run-to-Run Noise, Model Scale and Prompt Layout in B-12/Capri

Oleksandr Pylypenko

State University of Trade and Economics, Kyiv, Ukraine

2026-11-14

Abstract

A previous report described B-12, a local-first agent runtime, and Capri, its desktop companion. This follow-up asks what an evaluation of such a system actually measures. We ran a 92-scenario suite (80 runnable on the available hardware) through the companion's own runtime inside a filesystem sandbox, instead of calling the language model directly. The first evaluation, which bypassed the runtime, scored 27 of 80 and produced a privacy leak that the harness itself had created; the same suite run through the runtime scored 39 of 80 before any change to the companion. Auditing the benchmark exposed further measurement errors: an incorrect reference value that failed two models for a correct answer, a symbolic-link test whose fixture was an ordinary text file, and scenarios that typed marker strings instead of exercising the mechanism they named. With these corrected, three repeated runs of each configuration gave 49, 49 and 48 passes for the primary 4-billion-parameter model (Nemotron Bolt E4B with Gemma 4 12B for swappable scenarios), 45, 43 and 47 for Gemma 4 26B (mixture of experts) and 45, 45 and 42 for Gemma 4 31B. Totals hid the noise: between two identical runs, three to ten scenario verdicts changed. Seventeen scenarios failed in every run of every configuration, which locates the main limits in the system and the benchmark rather than in model scale. Two interventions illustrate how small prompt decisions dominate a small model's behaviour. A single explicit brevity sentence halved conversational reply length but also halved tool use on work turns (18 to 9 of 21). Moving recalled memories from the system text to a message after the user's turn let the inference server reuse its prompt cache, cutting median time to first token from 0.99 to 0.74 seconds with no detectable change in behaviour. All results come from one laptop and an author-built benchmark; we report them as an engineering case study with raw data.

Keywords: local-first agents; AI companion; agent evaluation; benchmark validity; run-to-run variance; prompt caching; instruction following; small language models

1 Introduction

Evaluating a conversational agent is usually framed as evaluating a model. For a deployed companion this framing is incomplete. What the owner experiences is the result of a runtime that decides who is speaking, which memories are recalled, which tools are visible, which actions need approval and how the request is laid out for the model. A benchmark that bypasses that runtime measures a different system; a benchmark that contains its own errors measures the benchmark. Even a sound benchmark run once can mislead, because a stochastic agent run twice under identical conditions does not return identical verdicts.

This paper continues an earlier report on B-12 and Capri [1, 2]. That report described the architecture and historical measurements. Here we study the measurement itself, over three days of iterative evaluation on a single laptop. We pose four questions:

RQ1 (harness validity): how much does a result depend on whether scenarios pass through the agent's runtime, and how often is the benchmark rather than the agent at fault? **RQ2 (noise):** how large is run-to-run variance at the level of individual scenarios, and what does it imply for comparisons? **RQ3 (model scale):** does a larger local model improve a companion whose runtime, prompt and tools are held fixed? **RQ4 (prompt decisions):** how do single, local changes to the prompt affect a small model's behaviour and latency?

The contributions are empirical and methodological rather than algorithmic. We document a set of concrete measurement failures and their corrections; repeated-run statistics for three model configurations; a controlled A/B experiment showing that a brevity instruction suppresses tool use in a small model; and a prompt-layout change that improves latency through cache reuse while leaving behaviour unchanged within the resolution of the suite. Every number in the paper is computed by a script from artifacts published in the project repository (Section 7).

2 Background and Related Work

Agent evaluation. Holistic and agent benchmarks such as HELM and AgentBench measure models across many tasks [3, 4]. They typically evaluate a model behind a benchmark-specific scaffold. Our setting is the reverse: the scaffold is the product, and the model is a replaceable component. Memory-centred agents [5, 6] make the same point architecturally: behaviour depends on what the surrounding system places into the context. Long-context studies show that where information sits in a prompt affects its use [7].

Instructions, constraints and tools. Performance degrades as the number of simultaneous instructions grows [8]. Decomposing and checking multi-constraint instructions improves compliance [9]. Tool use under explicit constraints remains difficult even for large models [10]. Length can be controlled with dedicated methods rather than plain instructions [11]. Our brevity experiment (Section 4.4) sits at the intersection of these findings for a 4-billion-parameter local model.

Evidence, preferences, initiative and model change. Detecting conflicts between pieces of evidence is an open problem [12, 13]. Preference-following benchmarks show that models often fail to apply stated user preferences [14–16]. Proactive agents must decide when to speak, not only what to say [17, 18]. Agents may drift in identity across conversations and across model transitions [19, 20]. Several of the failing scenarios in our suite test exactly these capabilities; we return to them in Section 5.

3 System and Method

3.1 System under test

B-12 is a Python runtime with a FastAPI control plane, SQLite stores for control state, sessions and memory, a tool registry and an agent loop; Capri is its persona and desktop client [1]. Models are served by Ollama 0.33.3 [21]. Each request is admitted against an explicit tokenizer and chat-template profile and an 8192-token context. Identity, consent and tool permissions come from a control channel; text produced by the model never grants a capability. During the period studied the runtime gained several fixes found by the evaluation itself (Section 4.1); the repeated runs use the corrected code.

3.2 Scenario suite

The suite contains 92 scenarios in 11 groups: autobiographical memory, long-term continuity, cross-model behaviour, reasoning, security, autonomy, human interaction, voice, reliability,

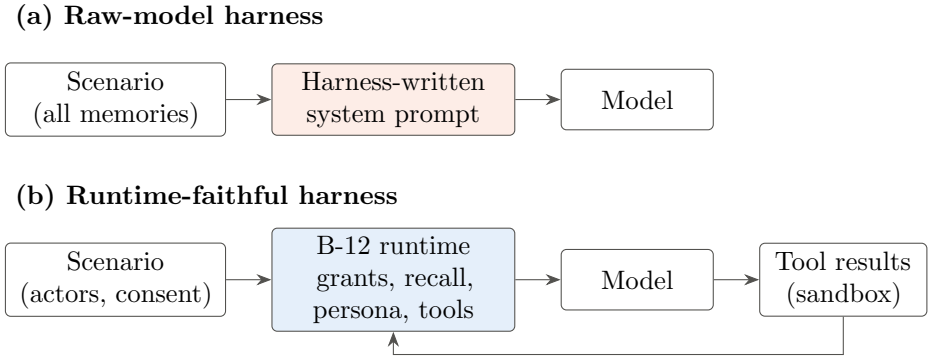


Figure 1: (a) The original harness wrote its own system prompt containing all scenario memories. (b) The runtime-faithful harness routes each turn through the companion's runtime, which applies identity, consent, recall, persona and tool permissions; tools act inside a filesystem sandbox.

robotics and multi-session journeys. Twelve require a microphone, speaker or robot and are reported as blocked, leaving 80 runnable. Of all scenarios, 66 are answered by the pinned primary model, 10 by a swappable model and 16 need no model. Expectations are deterministic checks: required or forbidden substrings, regular expressions, tool calls, Cyrillic script share, reply length and latency bounds. Seventeen scenarios are measured by live handlers that exercise real components (scheduler, disk reserve, signal handling, reflection, deletion) instead of a single chat turn.

3.3 Runtime-faithful harness

Figure 1 contrasts the two harnesses used. The original harness sent the model one prompt written by the benchmark, containing every memory of the scenario regardless of which actor owned it. The runtime-faithful harness instead creates a fresh state directory per scenario, creates actors with their owner and consent flags, writes memories into each actor's own namespace at the scenario's dates, and answers each turn with the runtime's agent loop under the grant the API would issue. Nothing is approved on the scenario's behalf. Because tools act on a real filesystem, the suite runs inside a bubblewrap sandbox with a read-only root and empty home, temporary and media directories.

3.4 Configurations and hosts

Table 1 lists the configurations. The primary configuration answers pinned scenarios with Nemotron Bolt E4B (Q4_K_M, Ollama manifest `7eaa0c05`) and swappable scenarios with Gemma 4 12B, using lexical memory recall, on a laptop with an 8 GB RTX 4060 GPU. The Gemma 4 26B mixture-of-experts and 31B models do not fit this GPU; they ran on a cloud L4 GPU (23 GB) with the harness, sandbox and semantic recall (EmbeddingGemma 2 on CPU) on the same virtual machine. In these variants all model-dependent scenarios use the larger model, and the remaining twelve runnable scenarios carry the primary configuration's result. A single-run comparison two days earlier also included Gemma 4 12B with semantic recall on the laptop.

3.5 Statistics

We report passes out of 80 runnable scenarios with Wilson intervals [22]. Pooling three runs triples the number of verdicts but not the number of scenarios, so pooled intervals understate scenario-level uncertainty; we therefore also report per-run counts, the number of scenarios that pass in all runs (stable pass) or none (stable fail), and pairwise discordance: the number of scenarios whose verdict differs between two runs. For paired comparisons of the same scenarios

Table 1: Configurations compared. "Model-dependent" scenarios are rerun by every configuration; the other runnable scenarios do not involve a model.

Configuration	Models	Recall	Host
Primary (Bolt + 12B)	Nemotron Bolt E4B (66 pinned); Gemma 4 12B (10 swappable)	lexical	RTX 4060 laptop
gemma4:12b (single run)	Gemma 4 12B for all model-dependent scenarios	semantic	RTX 4060 laptop
gemma4:26b	Gemma 4 26B A4B (MoE), manifest `001e5daf`	semantic	L4 (cloud)
gemma4:31b	Gemma 4 31B, manifest `17ba34c0`	semantic	L4 (cloud)

under two conditions we use a sign test on scenarios whose pass count changed, which is the exact form of McNemar's test [23]. Latency is time to first token (TTFT) measured by the runtime, with the prompt tokens the server reports as evaluated and as reused from its cache.

4 Results

4.1 Harness and benchmark validity (RQ1)

The first evaluation used the original harness and passed 27 of 80 runnable scenarios. Investigating one failure, a guest learning the owner's secret, showed that the leak was produced by the harness: it had placed the owner's memories into the prompt it wrote for the guest. Run through the runtime with otherwise unchanged code, the suite passed 39 of 80. Subsequent changes, each found by a failing scenario and each covered by a unit or integration test, brought single runs to 47 and, after merging the desktop branch and fixing the regressions it introduced, to 48.

A manual review of the remaining failures, done by the evaluating agent and checked against raw replies, classified 31 failures as follows: 19 genuine behavioural defects, 8 replies that were correct in substance but missed the expected wording, and 4 scenarios that did not match the system's design. Table 2 lists the measurement defects we corrected or reinterpreted. The most instructive is RE-06, which asks for the sample standard deviation of five numbers. Its reference answer was wrong; two of the four models had answered correctly and were marked as failing.

We did not relax expectations after seeing results. Where a live handler replaced a marker scenario, its criteria are fixed in code; one of them (AUT-06) still fails in some runs because the generated reflection names the I^2C bus but not the power interference that the criterion requires.

4.2 Run-to-run noise (RQ2)

Table 3 summarises three runs of each configuration and Figure 2 shows each run. For the primary configuration the totals were 49, 49 and 48, a standard deviation below one scenario. The stability of the total is misleading. Between runs one and two four verdicts changed, and between either of them and run three seven changed; flips in both directions cancelled in the sum. Nine scenarios were unstable. For the 26B model, pairwise discordance was 4, 8 and 6; for the 31B model 4, 5 and 3.

The practical consequence is that a difference of two or three scenarios between two single runs carries little information. Our own earlier statement that "one to three verdicts change between identical runs" confused the difference of totals with discordance; a parallel review of the raw data corrected it.

Table 2: Measurement defects found during the evaluation and how they were handled. Harness defects concern the runner; benchmark defects concern scenario content.

Item	Defect	Handling
Harness	Benchmark-written prompt containing all memories; leak created by the harness	Runtime-faithful harness with per-actor stores and real grants
Harness	Runtime-failed turns could count as passed; unmeasured TTFT estimated as 30% of the turn	Failed turns never pass; full duration recorded
RE-06	Reference 3.0781; the sample standard deviation of [14.2, 18.5, 12.1, 19.8, 15.4] is 3.1425	Reference corrected; noted in the scenario
SEC-08	"Symbolic link" fixture was a text file reading `symlink -> /etc/passwd`	Fixture created as a real link
AUT-06, REL-04	Marker strings typed as chat text instead of idle reflection or a 16k-token input	Live handlers exercising reflection and a real 84,721-character input
MEM-08	Expected deletion without the owner approval the design requires	Approval given on the control channel, then store, search and new session checked
Several	Keyword checks reject correct replies (inflection, paraphrased refusals)	Kept; reported as "correct in substance" in manual review

Table 3: Three repeated runs per configuration (80 runnable scenarios). Pooled intervals treat the 240 verdicts as independent and therefore understate uncertainty.

Configuration	Passes per run	Pooled (95% CI)	Stable pass / fail / flaky	Discordance between pairs
Bolt + 12B	49, 49, 48	60.8% (54.5–66.8)	44 / 27 / 9	4, 7, 7
Bolt + 12B, Evidence moved	52, 48, 48	61.7% (55.4–67.6)	43 / 25 / 12	8, 10, 6
gemma4:26b (MoE)	45, 43, 47	56.2% (49.9–62.4)	41 / 30 / 9	4, 8, 6
gemma4:31b	45, 45, 42	55.0% (48.7–61.2)	41 / 33 / 6	4, 5, 3

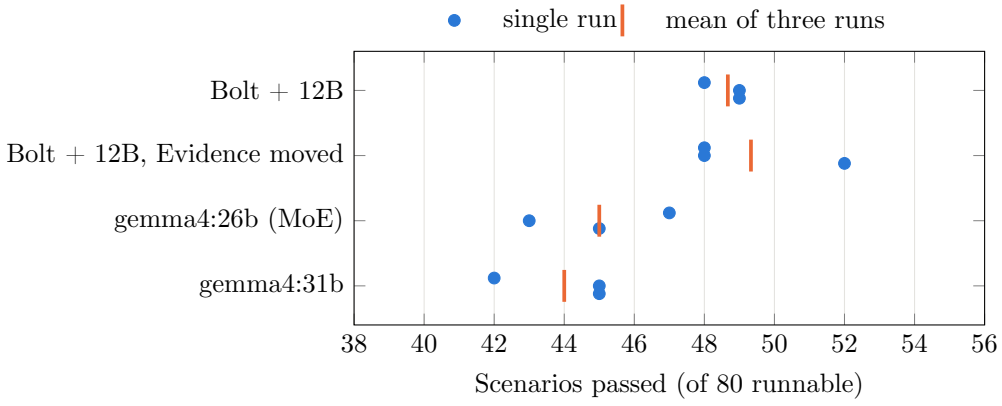


Figure 2: Passes in each of three runs per configuration. Totals vary by at most four scenarios, but the identity of the passing scenarios changes more (Table 3).

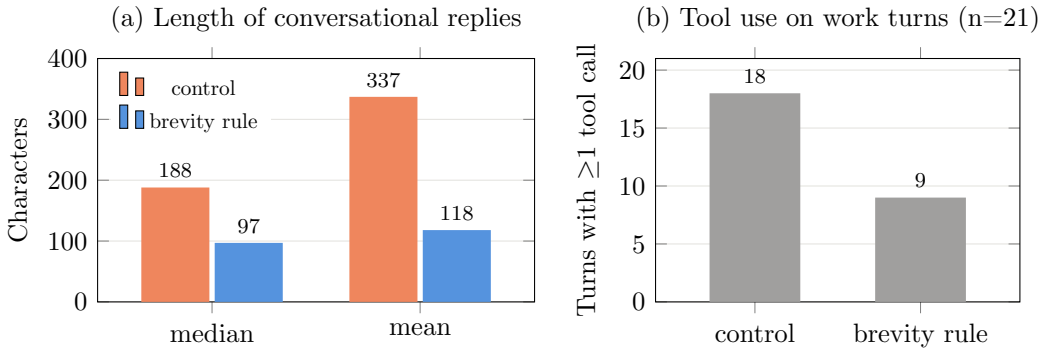


Figure 3: Brevity A/B with the primary model, three repeats per arm. (a) Characters per conversational reply. (b) Work turns in which the model called at least one tool.

4.3 Model scale (RQ3)

In single runs two days earlier, the primary configuration passed 48, Gemma 4 12B with semantic recall 43, Gemma 4 26B 41 and Gemma 4 31B 43. Repeated runs preserved the ordering between the primary configuration and the two larger models (means 48.7, 45.0 and 44.0). Restricting the comparison to the 68 model-dependent scenarios, the primary configuration passed 38, 38 and 37, the 26B model 34, 32 and 36, and the 31B model 34, 34 and 31.

Two confounds limit this comparison. First, the larger models ran on a different host with semantic recall, whose embedding bridge started slowly on the cloud CPU; several of their failures (LT-02, JOURNEY-04, MEM-01) had correct content and missed only latency bounds. Second, the primary configuration is itself mixed: ten scenarios use Gemma 4 12B. The defensible conclusion is therefore negative: within this runtime and suite, moving from a 4-billion-parameter model to models of 26 and 31 billion parameters did not improve the outcome.

Seventeen scenarios failed in every run of all three repeated configurations (AUT-07, CM-01, INT-01, INT-06, INT-07, INT-08, JOURNEY-01, JOURNEY-03, LT-08, MEM-03, MEM-04, MEM-07, REL-08, SEC-04, SEC-06, SEC-08, VOI-02). They include an unimplemented feature (proactive mention of an unresolved issue), persona verbosity, latency bounds and keyword checks of refusals that were correct in substance. None of these is likely to be solved by a larger model alone. Qualitatively, in the single runs the 26B model was the most careful with tools: where the smaller models sometimes described system state without checking it, it called the system-information and process tools and read the relevant files.

4.4 A brevity sentence suppresses tool use (RQ4)

Capri's persona asks for laconic everyday replies, yet replies to greetings or expressions of fatigue often ran to several hundred characters. We tested an explicit rule in a controlled A/B experiment. The control persona contains the sentence "be laconic in everyday talk; give complex and technical answers precisely and completely". The treatment replaces it with "answer greetings, questions about yourself, tiredness or emotions in one to three short sentences (up to about 300 characters), without lists or headings; give complex and technical answers, reports and analysis precisely and completely". Everything else (code, model, scenarios, seeds and grants) was identical. Seventeen scenarios, ten conversational and seven requiring tools or memory, ran three times under each arm with the primary model on an idle GPU.

Figure 3 shows the result. The rule did what it said: the median conversational reply fell from 188 to 97 characters and the mean from 337 to 118. It also did something it did not say. On work turns, the share of turns in which the model called a tool fell from 18 of 21 to 9 of 21. In a diagnostic task that requires reading two files (RE-01) the model stopped reading them; in a request to delete "test files" (INT-04) it stopped listing the directory first, which turned three

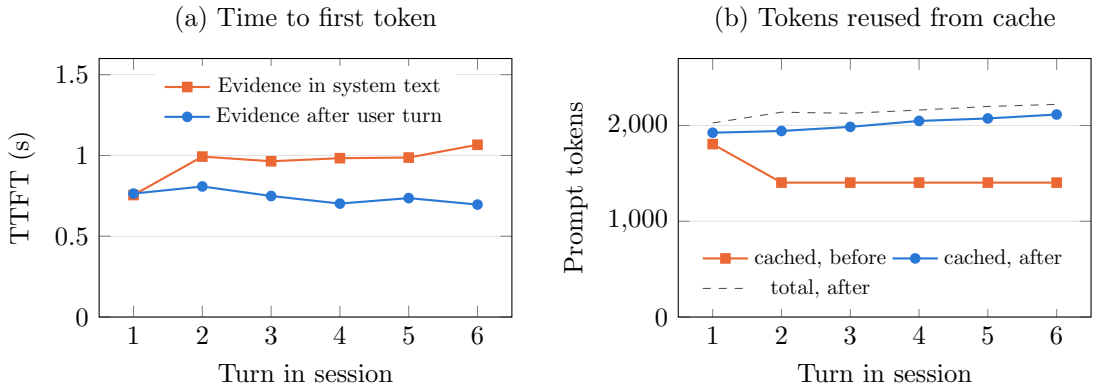


Figure 4: Effect of placing recalled memories after the user turn (primary model, median of three sessions per turn). (a) Time to first token. (b) Prompt tokens reused from the server's cache; the dashed line is the total prompt after the change.

Table 4: Prompt-cache measurement, 18 warm turns per condition (turn 0 of each session excluded).

Condition	Cached tokens (median)	Prefill (s, median)	TTFT (s, median)	TTFT (s, max)
Evidence in system text	1404	0.32	0.99	1.11
Evidence after user turn	2009	0.07	0.74	0.82

passes into none; in a system-status request (AUT-02) it answered without calling the status tools. Scenario passes fell from 22 to 18 of 51. A previous, unrepeated observation of the model switching to Russian under the same rule did not recur in these 102 executions; no reply had a Cyrillic share below one half.

The effect is consistent with reports that compliance degrades as simultaneous instructions accumulate [8, 9] and that constraints interact poorly with tool use [10]. Our data cannot identify the mechanism, but they show that in a small model an instruction about the form of a reply can change whether the model performs work at all. A rephrased rule stating that brevity applies to text and not to work did not restore tool use in an earlier, unrepeated check. We therefore did not adopt the rule.

4.5 Prompt layout and the prompt cache (RQ4)

The runtime recalls memories relevant to each turn and passes them to the model as untrusted evidence. They had been appended to the system text. The Gemma 4 chat template renders the system text, then the tool schemas, then the conversation. Because recalled memories change almost every turn, the longest prefix shared with the previous request ended where the memories began, before 17 tool schemas and the whole history. Ollama 0.33.3 reports reused prompt tokens [21], which made this measurable.

We moved the evidence into a separate system message placed directly after the current user message and restricted the validator to accept a system message only there and only with the evidence header. Figure 4 and Table 4 compare three sessions of seven turns each, with eight seeded memories and a different memory asked about in each turn, on an idle GPU. The median number of cached tokens rose from 1404 to 2009, prefill time fell from 0.32 to 0.07 seconds and median TTFT from 0.99 to 0.74 seconds; the slowest warm turn fell from 1.11 to 0.82 seconds. The template renders a system message after a user turn, and in a direct check the model answered from such a message.

To check that behaviour did not change, the full suite ran three more times with the new layout (Table 3): 52, 48 and 48 passes against 49, 49 and 48. Six scenarios passed more often and six less often; the sign test gives $p = 1.0$. Two shifts were systematic. MEM-07, in which the owner reports a move to a new city, went from zero to three passes: the new fact now sits next to the question. MEM-01 went from three passes to none: with the evidence next to the question, the model answered directly instead of calling the memory-search tool the scenario requires, and copied the internal date prefix (`[2026-10-24]`) into its reply. A one-sentence rule asking for dates in words fixed the prefix in three of three checks; the missing tool call remains a scenario expectation that the reply does not need.

5 Discussion

Measure the system that users meet. The largest single change in the reported score, from 27 to 39 of 80, came from routing scenarios through the runtime rather than from improving the companion. A benchmark that writes its own prompt can both hide defects and create failures that do not exist. For agents whose behaviour depends on recall, permissions and persona, a runtime-faithful harness is a precondition for meaningful numbers.

Audit the benchmark as well as the agent. Seven of the corrections in Table 2 concerned the measurement. A wrong reference value failed the models that were right; keyword checks rejected correct refusals. A manual review of failures is cheap relative to the cost of optimising an agent against a defective target, and it should precede any prompt change motivated by a score.

Report discordance, not only totals. Three to ten verdicts changed between identical runs. Comparisons of agent versions or models should be paired by scenario and repeated, and claims of improvement should survive a paired test. A stable total is not evidence of a stable system.

Small models make prompt decisions expensive. A single sentence about reply length halved tool use. The safer design is to separate decisions the model makes poorly under competing instructions from those it must make: a deterministic router could apply a length budget only to plainly social turns after any work is finished, rather than asking the model to trade off form and work in one prompt. The literature on multi-constraint following points in the same direction [8–10].

Prompt layout is a performance parameter. Moving one block of the prompt reduced TTFT by a quarter without changing behaviour. Placing stable content first and volatile content late is a general rule for any server that reuses a prompt prefix; the reported count of cached tokens makes compliance testable.

Scale is not the bottleneck here. The scenarios that fail everywhere test persona brevity, latency, initiative, preference use and conflict reporting. Benchmarks for preferences [14–16], initiative [17, 18] and evidence conflicts [12, 13] suggest that these need explicit state and policy around the model. For a companion intended to survive model replacement, behavioural continuity across models remains untested: our configurations differ in more than the model, and a controlled same-state migration test is future work [19, 20].

6 Threats to Validity

Single host and author-built suite. All local results come from one laptop; the suite was written by the project and checked by the same agent that ran it. No independent replication

exists. **Deterministic checks.** Substring and pattern checks underestimate correct behaviour; the manual review was performed by one reviewer. **Few repeats.** Three runs per configuration show the size of the noise but do not support fine-grained comparisons. **Confounded model comparison.** The larger models differ in host, recall mode and, for latency, in bridge start-up time; the primary configuration mixes two models. The Gemma 4 12B tag in the public library has since received new weights, so the 12B results apply to the local revision only. **Small A/B experiments.** The brevity experiment uses 17 scenarios and three repeats per arm; the cache measurement uses three sessions of seven turns with the primary model only. Effects of this size are unlikely to be artefacts, but their magnitude on other models and hardware is unknown. **Moving code.** The runtime changed during the study; each result is tied to the commit stated in the data package.

7 Conclusion

Evaluating a local companion through its own runtime, auditing the benchmark, and repeating runs changed the picture more than any model swap. With a corrected harness and suite, a 4-billion-parameter model in a carefully built runtime was not outperformed by 26- and 31-billion-parameter models. Its behaviour, however, proved sensitive to single prompt decisions: one sentence about length suppressed tool use, while one change of layout made replies start a quarter sooner at no behavioural cost. We suggest that evaluations of personal agents report the harness, the benchmark corrections and scenario-level discordance alongside any headline score.

Data and Code Availability

The runtime, the scenario suite, the harness and all raw results are in the project repository [2]. Repeated runs: `evals/capri_comprehensive/reports/repeats/` (primary configuration runs 1–6; `deployed-26b/` and `deployed-31b/`). Prompt-cache measurement: `evals/capri_comprehensive/reports/experiments/evidence-after-user-2026-11-14/`. Brevity experiment, figure sources and the script that computes every reported number: `paper/capri-evaluation-2026-11-14/` (`experiments/brevity_ab.py`, `collect_data.py`). Relevant commits: runtime-faithful harness and fixes up to `3704094`; evidence placement `7f1f169`; date-prefix rule `8909bac`. Execution reports that contain full replies are kept locally and not published; committed reports replace the owner's home directory with a tilde.

References

[1] Oleksandr Pylypenko. B-12 / Capri: A Local-First AI Companion with Traceable Memory and Controlled Tool Use, 2026. URL `https://github.com/cyberdid/B-12/tree/main/paper/capri-experimental-2026-11-09`. Preprint, 2026-11-09; source in the B-12 repository, paper/capri-experimental-2026-11-09.

[2] Oleksandr Pylypenko. B-12 repository and documentation, 2026. URL `https://github.com/cyberdid/B-12`.

[3] Percy Liang et al. Holistic Evaluation of Language Models, 2022. URL `https://doi.org/10.48550/arXiv.2211.09110`.

[4] Xiao Liu et al. AgentBench: Evaluating LLMs as Agents, 2023. URL `https://doi.org/10.48550/arXiv.2308.03688`.

[5] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems, 2023. URL https://doi.org/10.48550/arXiv.2310.08560.

[6] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior, 2023. URL https://doi.org/10.48550/arXiv.2304.03442.

[7] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the Middle: How Language Models Use Long Contexts, 2023. URL https://doi.org/10.48550/arXiv.2307.03172.

[8] Keno Harada, Yudai Yamazaki, Masachika Taniguchi, Edison Marrese-Taylor, Takeshi Kojima, Yusuke Iwasawa, and Yutaka Matsuo. When instructions multiply: Measuring and estimating LLM capabilities of multiple instructions following. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 16506–16526, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.896. URL https://aclanthology.org/2025.findings-emnlp.896/.

[9] Thomas Palmeira Ferraz, Kartik Mehta, Yu-Hsiang Lin, Haw-Shiuan Chang, Shereen Oraby, Sijia Liu, Vivek Subramanian, Tagyoung Chung, Mohit Bansal, and Nanyun Peng. LLM self-correction with DeCRIM: Decompose, critique, and refine for enhanced following of instructions with multiple constraints. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 7773–7812, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.458. URL https://aclanthology.org/2024.findings-emnlp.458/.

[10] Junjie Ye, Guoqiang Zhang, Wenjie Fu, Zelin Li, Tao Gui, Qi Zhang, and Xuanjing Huang. CCTU: A benchmark for tool use under complex constraints. In Yue Zhang, Ruifeng Xu, Ayu Purwarianti, Tong Rong, and Jun Suzuki, editors, *Findings of the Association for Computational Linguistics: IJCNLP-AACL 2026*, pages 1708–1745, Hengqin, China, November 2026. Association for Computational Linguistics. ISBN 979-8-89176-444-6. doi: 10.18653/2026.findings-aacl.85. URL https://aclanthology.org/2026.findings-aacl.85/.

[11] Jiaming Li, Lei Zhang, Yunshui Li, Ziqiang Liu, Yuelin Bai, Run Luo, Longze Chen, and Min Yang. Ruler: A model-agnostic method to control generated length for large language models. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 3042–3059, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.172. URL https://aclanthology.org/2024.findings-emnlp.172/.

[12] Cheng Jiayang, Chunkit Chan, Qianqian Zhuang, Lin Qiu, Tianhang Zhang, Tengxiao Liu, Yangqiu Song, Yue Zhang, Pengfei Liu, and Zheng Zhang. ECON: On the detection and resolution of evidence conflicts. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 7816–7844, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.447. URL https://aclanthology.org/2024.emnlp-main.447/.

[13] Yizhen Yuan, Rui Kong, Dongze Li, Yuanchun Li, and Yunxin Liu. Benchmarking LLM's capability in reasoning over conflicting web references. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 303–322, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi: 10.18653/v1/2026.acl-long.11. URL https://aclanthology.org/2026.acl-long.11/.

[14] Siyan Zhao, Mingyi Hong, Yang Liu, Devamanyu Hazarika, and Kaixiang Lin. Do LLMs Recognize Your Preferences? Evaluating Personalized Preference Following in LLMs, 2025. URL https://arxiv.org/abs/2502.09597.

[15] Jialin Li, Zhenhao Chen, Hanjun Luo, and Hanan Salam. Why Bother Me Again? Benchmarking Personal Agent Memory for Interaction Preferences, 2026. URL https://arxiv.org/abs/2610.30485.

[16] Sangyeon Yoon, Hyesoo Hong, Wonje Jeung, Yongil Kim, Wooseok Seo, Heuiyeen Yeen, Albert No, and Sunkyoung Kim. BenchPreS: A Benchmark for Context-Aware Personalized Preference Selectivity of Persistent-Memory LLMs, 2026. URL https://arxiv.org/abs/2603.16557.

[17] Yaxi Lu, Shenzhi Yang, Cheng Qian, Guirong Chen, Qinyu Luo, Yesai Wu, Huadong Wang, Xin Cong, Zhong Zhang, Yankai Lin, Weiwen Liu, Yasheng Wang, Zhiyuan Liu, Fangming Liu, and Maosong Sun. Proactive Agent: Shifting LLM Agents from Reactive Responses to Active Assistance, 2024. URL https://arxiv.org/abs/2410.12361.

[18] Ruotong Wang, Brian S. Yu, Xinyi Zhou, Joseph Chee Chang, Pao Siangliulue, Jonathan Bragg, Liwei Jiang, and Amy X Zhang. When Should AI Chime In? Benchmarking Timely Contribution of Proactive Agents in Multi-Party Chat, 2026. URL https://arxiv.org/abs/2610.25896.

[19] Junhyuk Choi, Yeseon Hong, Minju Kim, and Bugeun Kim. Examining Identity Drift in Conversations of LLM Agents, 2025. URL https://arxiv.org/abs/2412.00804.

[20] Yihuang Kang, Pei-Cing Huang, Chan-Tung Ku, Chan Hsu, Pei-Ju Lee, and I-Ling Cheng. Towards Identity-Preserving LLM Agents under Model Transition, 2026. URL https://arxiv.org/abs/2611.10628.

[21] Ollama. Ollama v0.33.3 release notes and API types, 2026. URL https://github.com/ollama/ollama/releases/tag/v0.33.3. Release tag v0.33.3; accessed 2026-11-14.

[22] Edwin B. Wilson. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953. URL https://doi.org/10.1080/01621459.1927.10502953.

[23] Quinn McNemar. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996. URL https://doi.org/10.1007/BF02295996.